| **Name of Student :** Raghav Mahesh Sarda | **PRN:** 25070122***235*** |
|---|---|
| **Subject: Programming with Java** | **Semester: III** |
| **ASSIGNMENT No : 3** | **Year: AY 2026-27** |
| **TITLE : Method Overloading and Static Members** | |

## Problem Statement

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

## Learning Objectives

To implement method overloading and use static variables and methods in Java programs.

## Program:

***Calculator.java***

```
// Assignment 3 - Exercise 1: Calculator using overloaded methods and static variable

import java.util.*;


public class Calculator

{

     static int calcCount = 0; // Static variable to count total calculations

     public static void add(int a, int b) // Overloaded method for integers

   {

       calcCount++; // Incrementing count

       System.out.println("Integer Sum: " + (a+b));

   }
```

```
    public static void add(double a, double b) // Overloaded method for
decimals
    {
        calcCount++; // Incrementing count
        System.out.println("Decimal Sum: " + (a+b));
    }
    public static void main(String args[]) // Main Method
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Two Integers: ");
        int i1 = sc.nextInt();
        int i2 = sc.nextInt();
        add(i1, i2); // Calling integer addition

        System.out.println("");
        System.out.print("Enter Two Decimals: ");
        double d1 = sc.nextDouble();
        double d2 = sc.nextDouble();
        add(d1, d2); // Calling decimal addition

        System.out.println("Total Calculations Performed: " + calcCount);
        sc.close();
    }
}
```

***RestaurantBilling.java***

```
// Assignment 3 - Exercise 2: Restaurant Billing using overloaded methods and static variable

import java.util.*;

public class RestaurantBilling

{

   static int totalorders = 0; // Static variable for tracking orders

    public static void CalBill(double dinebill) // Function To Calculate Bill For Dine-In

   {

       totalorders++; // Incrementing order count

           System.out.println("Dine-In Bill Amount = " + dinebill); // Displaying bill

   }

     public static void CalBill(double takebill, double charges) // Function To Calculate Bill For TakeAway

   {

       totalorders++; // Incrementing order count

       double finalbill = takebill + charges; // Calculating total

         System.out.println("Take-Away Bill Amount = " + finalbill); // Displaying bill

   }

     public static void CalBill(double deliverybill, double charges, double deliverycharge) // Function To Calculate Bill For Delivery

   {

       totalorders++; // Incrementing order count

         double finalbill = deliverybill + charges + deliverycharge; // Calculating total

          System.out.println("Delivery Bill Amount = " + finalbill); // Displaying bill
```

```
    }
    public static void main(String args[]) // Main Method
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("- Restaurant Billing -");
        System.out.print("Enter Dine-In Amount: ");
        double amt1 = sc.nextDouble();
        CalBill(amt1); // Calling overloaded method for Dine-In

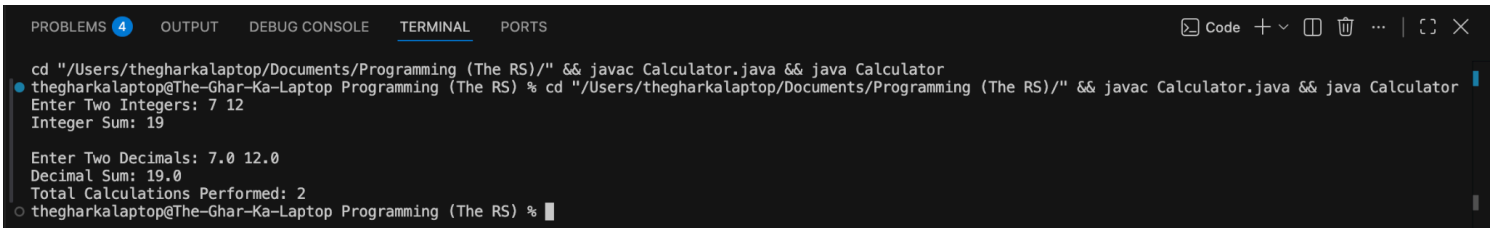
          System.out.print("\nEnter Take-Away Amount and Packing Charge:
");
        double amt2 = sc.nextDouble();
        double pack = sc.nextDouble();
        CalBill(amt2, pack); // Calling overloaded method for Take-Away

         System.out.print("\nEnter Delivery Amount, Packing, and Delivery
Charge: ");
        double amt3 = sc.nextDouble();
        double pack2 = sc.nextDouble();
        double del = sc.nextDouble();
            CalBill(amt3, pack2, del); // Calling overloaded method for
Delivery

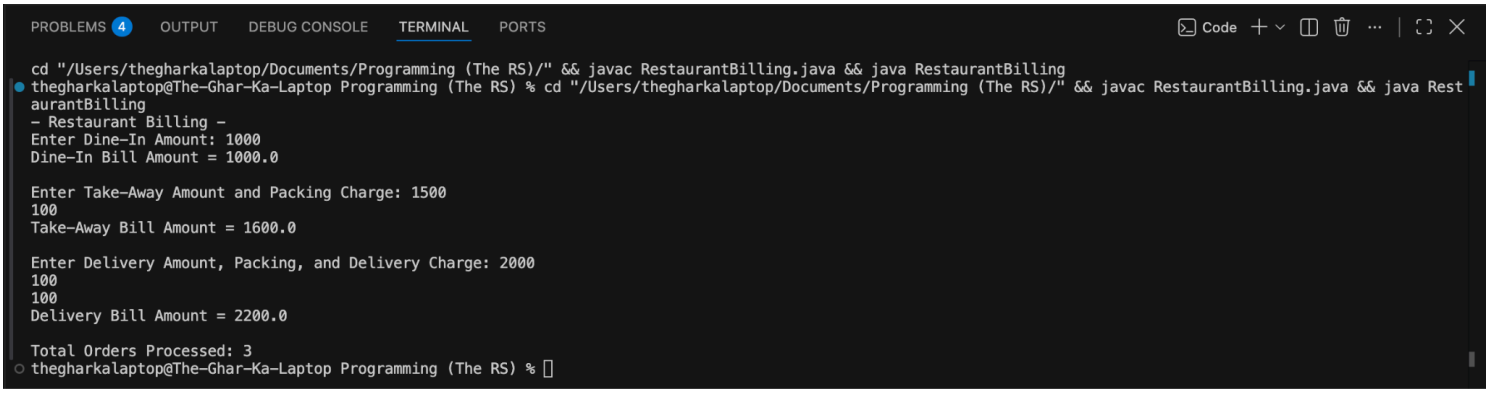
          System.out.println("\nTotal Orders Processed: " + totalorders);
// Displaying static variable
        sc.close();
    }
}
```

## Output:

### *Calculator.java*




### *RestaurantBilling.java*




## Conclusion:

In this assignment, the concepts of compile-time polymorphism and shared class members using method overloading and static variables were successfully implemented and verified. Method overloading provided flexible function calls with different parameter types, and static variables allowed efficient tracking of shared data across the entire class without requiring separate object instantiation.